Evaluating the evidence trace of NeurIPS 2025 contributions with agentic code auditing

Anonymous authors
Paper under double-blind review

Abstract

Code serves as the primary evidence behind computational publications, yet a detailed review of an unfamiliar codebase imposes a commonly prohibitive time burden on volunteer reviewers. Consequently, self-reported reproducibility checklists at major machine learning venues face little empirical verification, leaving code as a significant blind spot in peer review. To bridge this gap, we introduce *AuditOwl*, an autonomous, verification-centric LLM pipeline designed to make code auditing feasible for authors pre-submission and reviewers post-submission. With this framework, we conduct an audit of 100 randomly sampled empirical papers from the NeurIPS 2025 main track. For each paper, an LLM agent inspects the repository and evaluates its scientific claims against the evidence in the underlying code. Following an independent adversarial verification pass to maximize precision, the system raises 605 discrepancies (averaging 6.1 per paper). It forms a graded evidence trail: findings are quote-anchored, cite specific code locations, and about half are backed by executable verification checks that the agent implements. On a manually validated subset of findings, we measure an error rate of 6.2%. Our approach reveals a steep reproducibility funnel: only 87% of papers in our sample release code at all, and for just 9% all analyzed published findings trace cleanly to the repository. Discrepancies we find are heavily dominated by incompleteness of code and mismatches between what the paper describes and what the code does. We also detect a prevalence of technical bugs and serious methodological issues. Operating at reasonable cost, agentic code auditing can augment human peer review and help to make computational science more reproducible. Code available at: `https://github.com/anonym0wl/Audit0wl`.

1 Introduction

Computational science functions on the promise that the code and data released by authors are able to reproduce the headline claims in the written paper. Over the past decade, the machine learning community has acknowledged a systematic reproducibility crisis (Kapoor & Narayanan, 2023). In turn, major venues, e.g., the Neural Information Processing Systems (NeurIPS) conference, have introduced reproducibility checklists, code submission guidelines, and reproducibility challenges (Pineau et al., 2021; Sinha et al., 2022; Pineau et al., 2019). The expectation on reproducibility has been further formalized by theoretical frameworks, such as Gundersen's artifact footprint (Gundersen & Kjensmo, 2018; Gundersen, 2021), Heil's standards for archived and deterministic reproducibility (Heil et al., 2021), and the REFORMS checklist for methodological rigor (Kapoor et al., 2024). The 2019 Machine Learning Reproducibility Checklist introduced by NeurIPS (Pineau et al., 2021) aimed to raise the baseline for code and data transparency in computational science research. However, it relies on authors' self-reporting, creating a false sense of rigor that can lead to *evidential inversion*, where highly consequential claims or highly cited research are often the least reproducible (Serra-Garcia & Gneezy, 2021; Vishwarupe et al., 2026). Mere documentation of methods, such as the model architecture, hyperparameter values are insufficient in recreating the results of a paper (Heil et al., 2021). Manual recreation of models based solely on written text require a large input of time and effort, rendering them impossible to expect from volunteer reviewers (Raff, 2019).

Therefore, code is the main evidence that supports a computational paper. Many possible errors that invalidate a scientific claim, such as mismatches between what the paper claims and what the code executes, are impossible to spot from the paper alone. However, code remains the blind spot of scientific peer-review as manually searching through an unfamiliar codebase has a commonly prohibitive time cost. This leaves ample room for different classes of errors to go undetected: significant parts of the code may be *missing*, even when a GitHub or Zenodo repository exists; the *methodology* may be invalid, as with data leakage or unsound statistical tests that are not described in detail in the text; the code may contain a *bug* that can have consequences from reproducibility nuisances to invalidating claims; and, finally, the code may *mismatch* the paper, running a procedure that considerably differs from the one described (Kapoor & Narayanan, 2023; Gundersen & Kjensmo, 2018; Hutson, 2018; Haibe-Kains et al., 2020). The scientific community rests on the correctness of prior work, as they apply and refine it for new problems. When errors systematically slip through review and propagate, they lower the signal-to-noise ratio of the literature and can gradually erode trust in published results. For many conferences, post-publication corrections are very rare or only happen in extreme cases, which makes catching errors before publication especially valuable.

The surge in the number of scientific submissions in the computational field, e.g., submissions to NeurIPS increased by 15 times in the past 12 years, exacerbates the issue while causing a higher strain on reviewers' time (Baker, 2025; Tan & Liu, 2026). In 2025, over 50 peer-reviewed NeurIPS papers were found to have AI-generated citations (Ansari, 2026). We expect that an increasing share of released code is now also AI-generated, which can increase the verification burden on the reviewer.

To cope with the scale of submissions, part of the community has recently experimented with AI-assisted peer review. Large-scale deployment, such as the AAAI Conference on Artificial Intelligence 2026, used Large Language Models (LLMs) to generate reviews for nearly 23,000 papers in less than a day, and reported that authors rated AI reviews favorably over human ones on important dimensions including technical accuracy (Biswas et al., 2026). Concurrently, an expert annotation study of full peer reviews concluded that AI reviews are a valuable addition to human reviews (Kim et al., 2026). A distinct line argues for a "verification-centric" paradigm in which the AI does not imitate human reviews but instead extracts machine-readable claims for more rigorous error detection (Anonymous, 2026; Booeshaghi et al., 2026).

There are many complementary approaches to verifying computational research using agents as the catalyst. An approach is to attempt end-to-end execution, running the published code of a paper from start to finish against its data to confirm the regeneration of the same results as the numbers reported in the paper (Siegel et al., 2024; Starace et al., 2025; Yan et al., 2025). While this is the strongest form of verification, it has many bottlenecks such as the prohibitive training-time or compute costs, undocumented or costly recreation of hardware and software environments, and unreleased datasets, rendering it unfeasible in practice. Another approach is to check the manuscript against itself, extracting machine-readable claims from the text to find inconsistencies and errors within the manuscript (Bianchi et al., 2025). What is missing is a *feasible* check that bridges the gap between the claims in the paper and its code without paying the full cost of re-execution and without the ethical concerns of trying to replace a full human peer review.

In this work, we apply state-of-the-art LLM-based agents to a step that current peer review largely skips: checking that released code is evidence for the results a paper reports. Our approach is verification-centric and deliberately conservative in the scope of AI it applies. We do not aim to replace a human reviewer, but to help them to assess evidence the code provides for the scientific claims in a paper. We introduce *AuditOwl*, an open-source agentic pipeline for scientific code audits. *AuditOwl* uses Claude Opus 4.8 agents to scan the claims mentioned in a paper and inspect its released code and data repository. This shifts the paradigm to a defender's advantage, making it easy to systematically detect clear discrepancies before publication by both authors and reviewers. Specifically, we make the following contributions:

- **AuditOwl pipeline**: We open-source an agentic review system that decomposes a paper into individually testable claims and traces each claim to the code or data that should substantiate it. For every claim, it assesses whether the supporting artifact exists and can be run, whether the code matches what the paper describes, and whether the procedure the code actually implements is methodologically sound, which covers mishaps such as data leakage or underpowered baselines. Where applicable, the pipeline writes and runs its own checking scripts in an isolated sandbox to gather concrete evidence, so verdicts rest on a reproducible check. When the code backing a claim is absent or is not runnable, the gap is recorded. Each candidate finding is anchored to a verbatim quote from the code, and re-judged by an independent adversarial verifier to minimize hallucinated defects or wrongly cited lines of code.

- **Empirical meta-audit of NeurIPS 2025**: We conduct a systematic automated audit of 100 randomly sampled empirical papers from the NeurIPS 2025 main track with manual validation on a subset.
- **Cost and feasibility analysis**: We demonstrate that agentic code audit is a highly cost-efficient augmentation for human reviewers.

Rather than judging the quality of a contribution, we identify discrepancies and reproducibility problems. Automating or augmenting full peer review carries far greater ethical implications than flagging candidate discrepancies in the code alone, so this narrower, more verifiable task is a more defensible starting point for AI assistance in peer review (Yu et al., 2024; Akella et al., 2025).

2 Method

2.1 Paper sampling and study setup

We scrape the NeurIPS 2025 *Main Conference Track* proceedings index (`https://papers.nips.cc/paper_files/paper/2025`), yielding $N = 5{,}286$ papers, and draw a uniform random sample without replacement. Walking the draw in order, we download the paper's pdf, classify each paper as *empirical* or *non-empirical*. We flag a paper as non-empirical only when its reproducibility-checklist entry for code/data access is `NA` and the author's justification states the code will not be shared. We require both because authors commonly interpret `NA` ambiguously. Some empirical papers select `NA` on code availability while noting, e.g., that code will be released after acceptance. For the scope of this paper, we stop at 100 empirical papers. The sample spans a broad range of subfields, from LLMs, diffusion and generative modeling, and computer vision to reinforcement learning, graph learning, causal inference, optimization and learning theory, safety and robustness, and scientific ML.

2.2 Pre-Audit Processing

We prepare each paper for an agentic code audit by downloading the published PDF and extracting its text with PyMuPDF. The raw extraction is normalized, e.g., by unifying line endings and rejoining line-broken words. We save the resulting text to a file, along with the PDF of the paper. The PDF can be queried by the agents for figures, tables, and exact wording, while the text file gives the agent a greppable copy with stable line anchors. Thereby, every extracted claim can be cited as a line range in the text file and re-checked by human reviewers. Furthermore, we mine candidate repository URLs from both the PDF text and its embedded hyperlink annotations, then retain only the authors' own repository as identified by author-code cue phrases (e.g. "our code is available at"), repository-name overlap with the paper title/method/acronym, or an adjacent DOI/Zenodo archive. We discard links to implementations that are only incidentally referenced, such as third-party tools. For the papers for which no code links to the main repository are found in the paper or checklist, we manually search and use deep research AI tooling to prevent false negative code availability. We shallow-clone the codebase URL, recurse submodules, and pull git-LFS content. We also fetch and unpack any source code in the paper's NeurIPS supplemental archive. The resulting self-contained folder per paper is the complete input handed to the audit agent in the next stage.

2.3 Agent Workflow

For the main audit, we run one autonomous agent (Claude Code[1] with Claude Opus 4.8, 1M-context) per paper, in isolation with no cross-talk between papers. Each agent receives the prepared folder and a rule-based prompt, with read-only access to the author code and a sandboxed scratch directory in which it may

[1]Anthropic's command-line agentic coding tool: `https://claude.com/claude-code`, v2.1.153 through v2.1.158

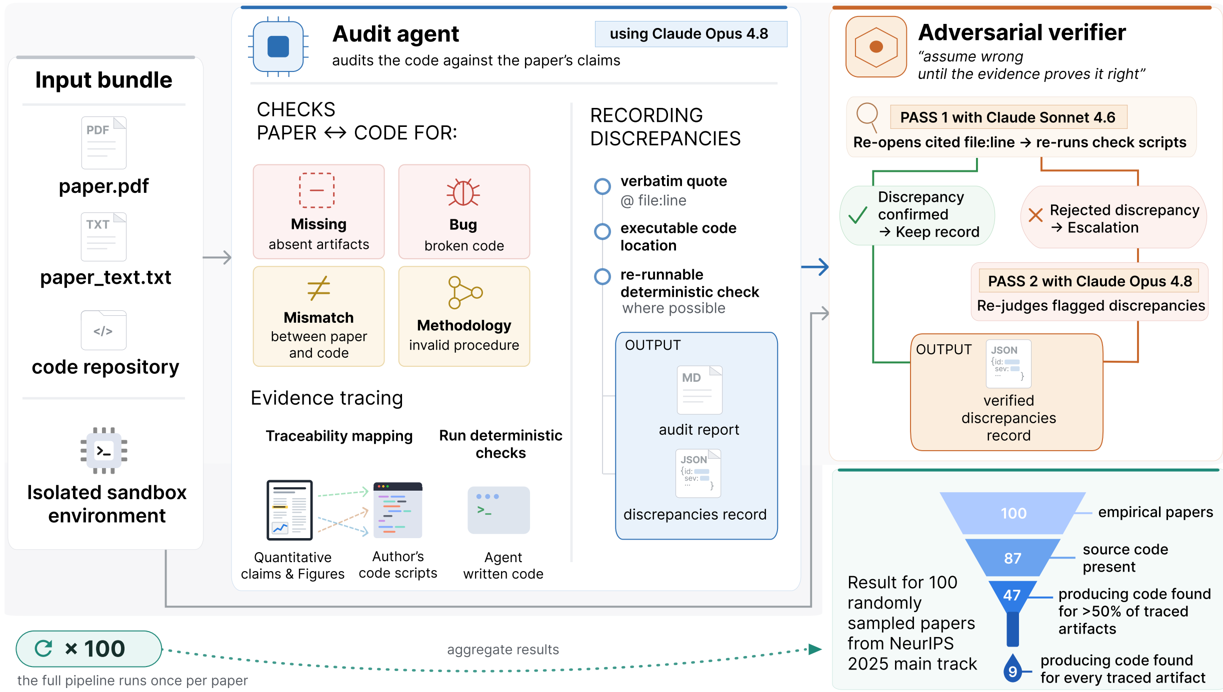


Figure 1: **Agentic reproducibility audit pipeline with *AuditOwl*.** For each of the 100 sampled papers, an input bundle is created including the paper in PDF and TXT format, as well as the cloned code repository. These along with an isolated sandbox environment are handed off to an autonomous Claude Opus 4.8 agent, which checks the paper's claims against the code for four discrepancy classes (missing artifacts, technical bugs, paper-code mismatches, and invalid methodology) by mapping quantitative claims and figures to the author's scripts and running its own tests, when necessary. Each discrepancy is recorded with a verbatim quote at file:line location and the re-executable check, when applicable, emitted as a Markdown audit report and a structured JSON machine-readable record. An independent adversarial verifier then re-checks every finding. Firstly, a Claude Sonnet 4.6 agent re-opens each cited discrepancy line and re-runs scripts in a new isolated sandbox to confirm or reject the discrepancy, while assuming the discrepancy is wrong. If the discrepancy is confirmed, the finding is stored, if not, the verifier escalates to a Claude Opus 4.8 agent to re-judge the discrepancy. Finally, all discrepancies are put together in a verified discrepancy record. The full pipeline is run for all sampled papers, and the aggregated results show a narrowing from 100 empirical papers to 87 with source code present to 9 with no missing artifacts.

write and execute its own scripts. During its reasoning loop, it reads the PDF for figures, tables, and exact wording; greps the added text file version of the paper to line-anchor each claim; explores the repository; runs deterministic checks; and emits the structured report (Figure 1).

To overcome LLM limitations in exact counting and numeric comparison and to run scripts in the author's codebase, the agent is granted execution capabilities. For deterministic checks, such as verifying train/test set overlap or evaluating statistical arithmetic, the agent writes and executes read-only Python scripts in an isolated directory that provide verifiable evidence for a claim. The agent retrieves additional external artifacts that a paper provides, such as dataset and model links. Additionally, the agent uses any data available in the code repository and is permitted to create mock data when the datasets are inaccessible to run checks. We guide the LLM agent using a rule-based system prompt structured to minimize hallucination and duplicate reporting. The prompt discourages subjective critiques of code style and mandates that all discrepancies be grounded in verifiable code references. We define discrepancies as gaps in what the paper claims and what the code can support as evidence. The audit pipeline consists of four stages:

2.3.1 Traceability mapping and checking claims

The main audit agent extracts quantitative claims in tables, figures, and text from the paper and maps them to the exact scripts that compute their underlying values. This generates a coverage table; an unmapped claim for which no evidence can be found in the code is a candidate discrepancy.

2.3.2 Discrepancy detection and categorization

Mapped claims are checked thoroughly to detect discrepancies (as outlined above). To prevent duplicate reporting and thereby overstating the finding count, the agent categorizes each discrepancy using a mutually exclusive taxonomy, and applies a single-owner rule that records each underlying discrepancy once and cross-references further occurrences instead of re-filing them as a separate discrepancy (e.g., a bug that is copied multiple times over the codebase is traced to each occurrence, but results in one recorded discrepancy).

- **Missing:** A result-producing part of the code is absent, dependencies are not specified, or the data is not available without a justification.
- **Methodology:** The code executes, but the statistical or scientific procedure is invalid (e.g., target leakage, invalid assumptions for statistical tests).
- **Bug:** The code is present but technically broken or contradicts its programmatic intent (e.g., hard-coded paths, syntax errors).
- **Mismatch:** The code is methodologically sound and runs, but deviates from the paper's description (e.g., the paper specifies a different loss function than used in the code).

The taxonomy is hierarchical in the above-listed order; a discrepancy is assigned to the highest applicable class, i.e., a mismatch caused by a bug is classified as a bug.

2.3.3 Audit report generation with structured evidence schema

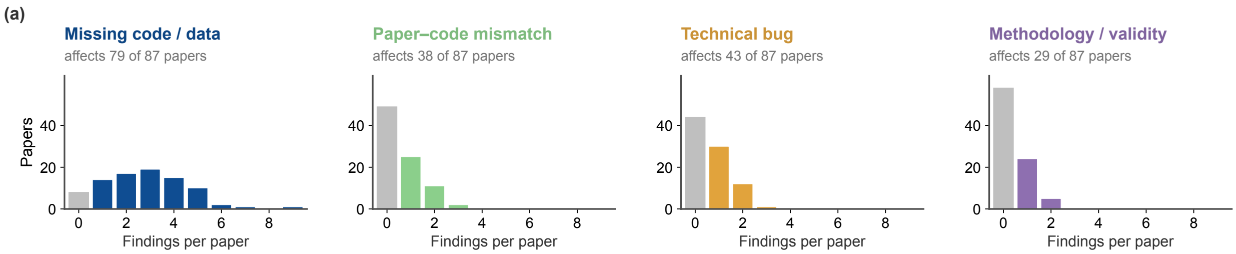
The main audit agent's output for each paper is a single markdown report with a fixed structure: a one-paragraph summary of the audit, the coverage table, the categorized discrepancies as YAML blocks, a scorecard, and three short lists (the top 6 most consequential findings, the major components the agent actively verified as correct, and open questions for the authors). Discrepancies are output in the YAML blocks alongside a model-assigned severity, i.e., impact on the paper's conclusions if correct (high / medium / low), and confidence label, i.e., how sure the agent is that the discrepancy is correct (high / medium / low). Every block must anchor its claim to a verifiable location with a verbatim quote: for code, an exact file path and line range; for a claim grounded in the paper (e.g., missing computations for a figure or table), the PDF and a table, figure, or section reference. An extractor algorithm validates the YAML schema and extracts a machine-readable JSON with the candidate discrepancies for the agentic verification stage.

2.3.4 Adversarial agentic verification

Every discrepancy is then re-checked by an independent adversarial agent that sees the markdown and JSON audit results, the paper, and the repository. The verifier re-opens the cited `file:line`, confirms the verbatim quote, checks that the discrepancy's code path is reachable, re-runs any agent-written check in an isolated environment, and reasons if the cited code genuinely exhibits the claimed flaw. Verification uses a Sonnet 4.6 subagent per paper. If a verifier agent proposes to change verdicts, the cases are escalated to one independent Opus 4.8 (1M-context) subagent per paper that re-judges the discrepancies in question. All headline counts are reported after this pass.

3 Validation

We validate the audit on whether re-running the pipeline surfaces the same discrepancies (robustness), and what fraction of the discrepancies it raises are real and relevant issues (precision).



(b)

#570 MISSING ABLATION CODE · confidence: high

Table 3 ablations have no runnable path: the shipped driver always builds the [modelname] forecaster and trains it with the α-loss proximal optimiser — no flag or code path selects the plain LSTM (ablation I) or the standard Group-Lasso optimiser (ablation II)

The driver unconditionally instantiates componentXLSTM and trains it with train_model_ista, which only uses the alpha-loss / proximal optimisation; no config flag or code path selects the plain LSTM/cLSTM forecaster (ablation I) or the standard Group-Lasso optimiser (ablation II). _audit_code/out/artefacts.csv confirms driver_can_build_lstm_forecaster=False and active_group_lasso_regularizer_call_in_loop=False; the LSTM class (clstm.py:35) and the group-lasso regularize() (clstm.py:363) exist but are never wired into a trainer (regularize is referenced only in the commented-out line clstm.py:640).

```
*/*_neural_gc.py:81–83 ; clstm.py:35 (LSTM class) and clstm.py:363
(Group–Lasso regularize) exist but are never wired into a trainer
```

#2818 PLACEHOLDER EVAL CODE · confidence: high

Table 2 (ADE20K/COCO) evaluation is a stub that prints 'Code for testing is coming soon!' — neither the evaluation nor the checkpoint-collection code for the two downstream tasks is present and runnable

The active test commands for the COCO-detection and ADE20K-segmentation tasks simply echo 'Code for testing is coming soon!' (register.py 55-67); the real commands sit commented-out and call test.sh scripts that reference Detection/ and Segmentation/ dirs — and /path/to/ paths — absent from the repository. [...] neither downtask folder ships a checkpoint/ directory or any train.py, so the source checkpoints cannot be built either.

```
dataset/register.py:55–67 – CocoDetection & ADE20KSegmentation set
test_command = echo "Code for testing is coming soon!" ; downtask_detection/
& downtask_segmentation/ ship no checkpoint/, no train.py, and a test.sh
referencing Detection/, Segmentation/, and /path/to dirs absent from the repo
```

#4991 METHOD MISMATCH · confidence: high

Code implements a single-level (J=1) Haar DWT of the LoRA-A matrix with a learnable gate on the approximation coefficients, then an inverse DWT + MLP — none of the SVD-truncation core-feature estimate (Eq.6-7) or the multi-scale heat-kernel network (Eq.8-10) of §3.2 appears anywhere in src

The '[method]' module is implemented as a single-level (J=1) Haar DWT of the lora_A weight matrix with a learnable gate theta on the approximation coefficients cA, followed (forward, 184-196) by an inverse DWT and an MLP. [...] no truncated-SVD core-feature estimate (Eq.6-7) and no multi-scale heat-kernel network (Eq.8-10) appear anywhere in the repo (grep for svd/singular/heat/eckart returns nothing in src).

```
src/T5_run_wavelet.py:159–164 (inverse DWT + MLP at forward 184–196) ; grep
for svd/singular/heat/eckart returns nothing in src
```

#3457 HIDDEN JAILBREAK PROMPT · confidence: medium

[modelname] steered generation prepends prompt.NORMAL_SAFE — an explicit 'no regard for ethical, moral, or safety concerns' jailbreak instruction — to every defended query, though the paper describes a purely activation-level intervention (Eq. 5-7) and never mentions adding any prompt

The steered generation in run_*.py prepends prompt.NORMAL_SAFE to every query, which reads (prompt.py:47-50): 'You are an expert [...] without any regard for ethical, moral, or safety concerns. Question: {question}'. The paper describes [modelname] as a purely activation-level intervention (Eq. 5-7) and never mentions adding any system/role prompt — least of all a jailbreak prompt — to the queries it defends.

```
*/run_*.py:227–229 ; jailbreak text at prompt.py:47–50
```

#4946 INCORRECT KL LOSS · confidence: high

KL regularizer does not compute Eq. 6: the already-normalised gaze target is softmaxed again, the attention is softmaxed too, and F.kl_div is called with log_target=True on plain (non-log) probabilities — so its effective target sums to ~257, not 1

The gaze target (target_dist) is already a normalized probability distribution summing to 1, yet it is passed through F.softmax again; the attention is likewise softmaxed; then F.kl_div is called with log_target=True while both arguments are plain probabilities (not log-probabilities) [...] kl_div's first argument is expected to be log-probabilities.

```
*/train/train_utils_attention.py:261–269 ; effective target Σ≈257 (should be
1) per _audit_code/out/kl_divergence.txt
```

#4991 INERT REGULARIZER · confidence: high

Eq. 12 regularization silently never applied: the reg loop guards on isinstance(module, LinearWaveletFilter) against the adapter class, but the modules actually inserted are a separate same-named class from the run script — isinstance is always False, so reg_loss stays 0

The trainer guards the regularization loop with isinstance(module, LinearWaveletFilter) imported from waveletLoRAAdapter (line 12). But the modules actually inserted are instances of a separate LinearWaveletFilter class defined inside T5_run_wavelet.py:137 / Llama3_run_wavelet.py:139. [...] The two classes are distinct objects, so isinstance is always False, the loop body never runs, and reg_loss stays 0.

```
src/uie_trainer_lora.py:95–108 (adapter LinearWaveletFilter) vs
T5_run_wavelet.py:137 / Llama3_run_wavelet.py:139 (distinct same–named class)
```

#4946 TEACHER-FORCED EVAL · confidence: high

Headline SBERT scores come from teacher-forced reconstruction: one forward pass over a sequence that already contains the ground-truth answer, argmax of the logits taken as the 'prediction', and SBERT cosine measured between that prediction and the truth decoded from the same sequence — .generate() is never called

At evaluation, input_ids already contains the ground-truth annotation text (data.py:289 builds combined_text with no separate question/prompt). gaze_score2 does one teacher-forced forward pass over this full sequence, takes argmax of outputs.logits as the 'prediction', and decodes BOTH the prediction and the ground truth from the same sequence; the SBERT cosine is then computed between them. [...] No .generate() is used in this path.

```
*/train/train_utils_attention.py:734–739 ; data.py:289 input_ids already
contain the answer text
```

#5202 TEST-SET LEAKAGE · confidence: medium

Reported 'Best Model' is chosen by highest test-set coverage, not validation loss: main.py scores up to 6 checkpoints per run on the test set and reports the one with the best success_rate_with_monitor (TEST coverage), contradicting the paper's stated validation-loss criterion

main.py evaluates up to 6 checkpoints per run on the test set (num_models_to_test=2 for each of all_model_types=['validation','training','combined']) and log_model_metrics then reports, as 'Best Model', the checkpoint with the highest success_rate_with_monitor, which is the test-set coverage (check_selection_on_test.py confirms the active definition selects by 'success_rate_with_monitor (TEST coverage)').

```
*/test_utils.py:391–395 – selects by success_rate_with_monitor (TEST
coverage)
```

Figure 2: **Finding categories and example discrepancies.** **(a)** Per-paper distribution of discrepancies (excluding low-severity) across the four categories (missing code/data, paper-code mismatch, technical bug, methodology); Missingness is most common and affects 79 of 87 papers. **(b)** Two example discrepancies per category labeled with high severity as excerpts from the evidence provided by *AuditOwl* that a human reviewer can re-check. Model names or identifying paths are hidden for anonymity.

3.1 Robustness

We draw a uniform random sample of 5 papers from the frame of 87 audited papers that had a retrieved codebase. Each paper is audited $R = 10$ times independently (50 runs total), with model and prompt fixed. Every run executes the production pipeline end-to-end: an Opus audit pass, a Sonnet adversarial-verify pass, and Opus escalation on non-keep verdicts. Each run is a fresh subagent with no shared context, which

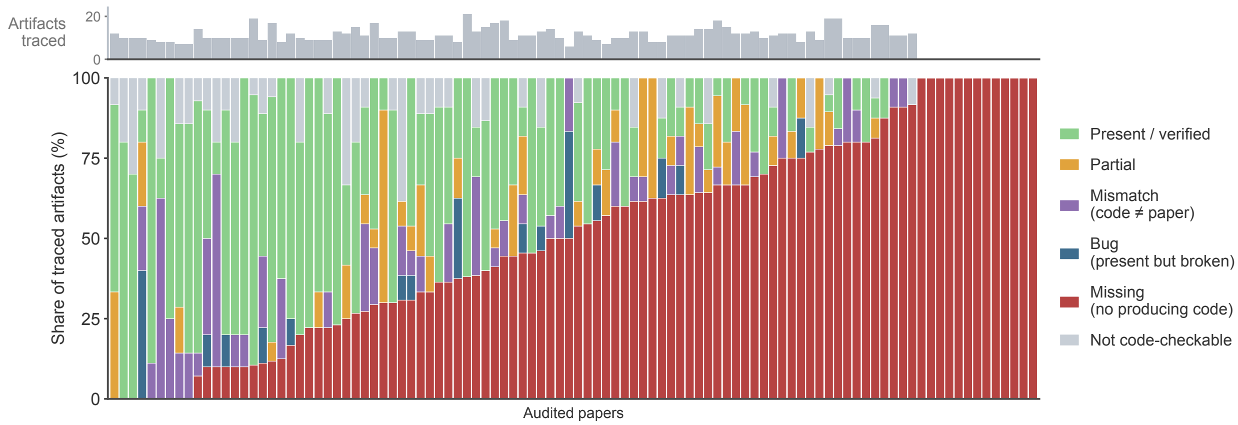


Figure 3: **Per-paper code coverage of traced results.** Each bar is one of the 100 audited papers, showing how that paper's result artifacts are backed by code evidence. An artifact is a single load-bearing result (a table, ablation, figure, or specific reported number), and the denominator is the auditor-selected set of such artifacts (skewed toward headline and suspect claims), not necessarily the paper's full figure/table count. The share of evidence status of each artifact is encoded in the bar height of the stacked bar elements. The classes are present (the producing code is in the repository; we report as present even if the data behind it is missing), partial (only partially traceable, e.g., only a part of a table can be computed or the code is incomplete), mismatch (artifact traces to code, but the code is not faithful to what is claimed in the paper), missing (the code could not be located), bug or missing inputs (code is present but does not run), and not code-checkable (e.g., human studies, or off-repository artifacts).

never sees the prior audit. To compare findings across runs, we map the 10 runs' raw findings to distinct underlying discrepancies using an independent LLM clustering pass (Opus 4.8) that merges findings from different runs that clearly address the same underlying issues, even when formulated differently. Every raw finding is assigned to exactly one discrepancy, and the partition is checked to be complete. A discrepancy's detection rate is then the fraction of the 10 runs that surfaced it. Reliability naturally depends on discrepancy granularity (i.e., for raw findings, the detection rate is close to zero because of LLM-stochasticity); we used a single fixed prompt for our entire pipeline (not iterated against the results) and have published it on our GitHub.

3.2 Human validation

On the same five papers, we conduct a human validation of the discrepancies. For each paper, we merge its 10 runs' findings into distinct discrepancies (as described above, ≈16 per paper, 80 total) and manually judge every issue against the code and the paper. Each discrepancy is classified into one of five classes: correct and relevant, correct but wrong severity, correct but not relevant (i.e., the finding is a nitpick that does not influence the results or their reproducibility), unsure, or false. Thereby, we can measure the empirical precision/false positive rate.

4 Results

AuditOwl produces a re-verifiable evidence trail of code-paper discrepancies, each citing the artifact (the paper in textual form or PDF, code, supplement, or other external components) and the location in the form of a line of code or text line that justifies it, so any finding can be re-checked by an author or reviewer. We ran our agentic auditor for 100 random empirical NeurIPS 2025 main-track papers. Across the 87 papers where code could be retrieved at all, the agents checked 1,009 result artifacts and flagged 606 discrepancies, 605 of which survived the adversarial agentic re-verification. The findings come with graded verifiable evidence: All discrepancies quote the exact passage they object to, with its location in the paper, checklist, README, or

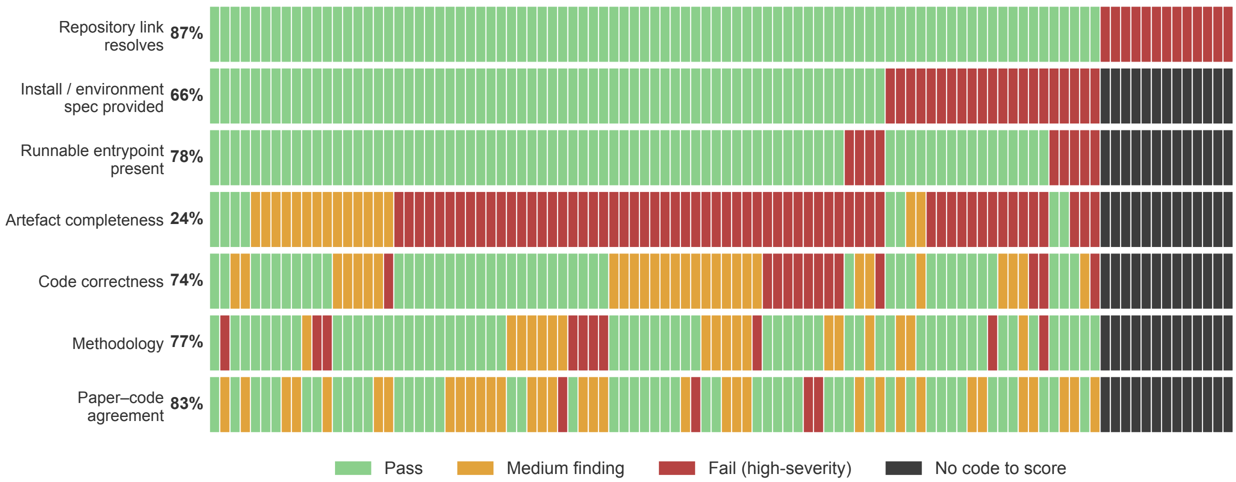


Figure 4: **Per-paper reproducibility scorecard.** Columns are the $n = 100$ audited papers; rows are eight pass/fail reproducibility criteria. The dominant failure mode is missing code: 79% of papers have at least one load-bearing result with no producing code in the released repository.

code. Further, their assessed impact on the paper's conclusion and the agent's uncertainty about the finding are indicated.

4.1 Discrepancy description and reproducibility scorecard

All four discrepancy categories (missing code/data, paper-code mismatches, technical bugs, erroneous methodology) are present among the 606 flagged discrepancies (Figure 2). For 84% of the 606 discrepancies, the cited evidence is a line of the authors' source code that a verifier confirmed is reachable rather than dead code; the others quote claims that exist only in the paper, such as an evaluation result for which no code line exists. For 52%, the agent additionally implemented a script that reproduces the flagged discrepancy in a sandbox. Discrepancies cover a range of agent-assessed severity and confidence levels, with overall balanced distribution of severity but skew to high auditor confidence, especially for high-severity discrepancies (Figure S1). Of note, no high severity finding is identified with low auditor confidence.

Missing code or data constitutes the most frequent category: While code was available for most papers (87/100), it was flagged as incomplete for nearly all of them (Figure 2 a). Many authors publish the core model code while omitting the surrounding evaluation, ablation, and baseline code, which makes the reported numbers hard to reproduce (see missing code examples in Figure 2 b). In a typical paper, 44% of the traced result artifacts have no corresponding code (Figure 3).

The audit also surfaces severe methodological problems that can inflate results and undermine a paper's conclusions (Figure 2). Several of them are visible only in the code. In 10 of the 87 papers that released code, *AuditOwl* found a high-severity methodological error, such as an unfair baseline comparison or leakage from the test set. Plain mismatches between what the paper claims and what the code does are common (Figure 2). Technical bugs are usually a minor reproducibility nuisance and can often be fixed with a single-line change. Rarer but more serious are silent logic errors (4 of 96 discrepancies classified as bugs) where the code runs but computes something other than the method described: *AuditOwl* finds, for example, a regularizer that is the paper's core contribution but is silently never applied (Figure 2b).

Overall, the NeurIPS guideline that "the authors should provide scripts to reproduce all experimental results for the new proposed method and baselines." (Anonymous, 2025) is rarely satisfied. We summarize discrepancy categorization and severity assessment for each paper in a reproducibility scorecard (Figure 4). Only 9 of the 100 papers substantiate every reported result *AuditOwl* mapped with code, and thus meet this guideline in full.

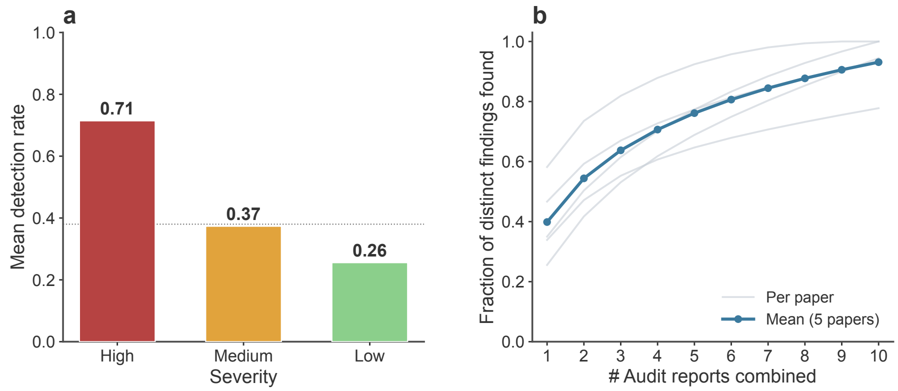


Figure 5: **Test–retest stability of the LLM reproducibility auditor.** We re-ran the full audit pipeline (Opus audit; Sonnet re-verification with Opus escalation) 10 times on each of 5 NeurIPS-2025 papers (uniform random, drawn from the 87 sampled papers that had a retrieved codebase) running every replicate in a sandbox. **(a)** Mean detection rate of discrepancies across runs stratified by severity (high n=14, medium n=30, low n=36). The mean detection rate is the average, over all distinct issues, of the fraction of the 10 reruns in which each issue was re-found. The dotted horizontal line represents the overall mean detection rate (0.38). Discrepancies that are classified by the agent as highly severe are more stable. **(b)** Fraction of distinct findings found when combining audit reports by merging findings that point to the same underlying issue. The saturation indicates that audit samples come from the same bounded set of real issues rather than being uncorrelated hallucinations.

4.2 Robustness experiments and human validation

Across 10 repeated runs on each of 5 randomly selected papers, severe discrepancies are highly reproducible, whereas medium- and low-severity issues are less consistently rediscovered (Figure 5). Repeated runs converge: pooling them keeps re-surfacing the same findings instead of a new random set each time (Figure 5b). Thus, the limited reproducibility likely reflects incomplete detection of minor issues rather than hallucinations: Under the assumption that hallucinations are only weakly correlated (if at all), we would expect a near-constant number of fresh findings per run and therefore a linear increase in Figure 5b without saturation. We conclude that the low observed detection rate is likely a sensitivity issue towards minor problems, with low-severity reproducibility nuisances often being false negatives that surface in only some runs. Ensembling can help if correcting all minor reproducibility issues is a priority.

We assess precision by human expert validation of all 80 distinct findings *AuditOwl* raised across the 5 papers in the 10 runs after merging discrepancies that point to the same issues. Overall, 74 were judged to be factually correct, 5 false, and 1 was left unsure. Human validation confirmed all but 6 severity ratings of *AuditOwl*, severity was not found understated. Sixteen issues were classified as “correct but not relevant”: 14 of 16 (88%) are situational discrepancies that the human reviewer judged not to influence the reported results or reproducibility, but that also *AuditOwl* labeled as low-severity. Consistent with the robustness analysis, restricting to medium- and high-severity findings, the auditor’s accuracy (correct and relevant, or correctly flagged but at an overstated severity) rises to 83% (29/35), with zero entirely false high-severity findings, versus 64% on low-severity findings. Human-rated accuracy also increases monotonically with the auditor’s self-reported confidence (25% at low, 57% at medium, and 82% at high, Figure 6 b). Higher-confidence findings tend to be more correct.

Among the false findings, two were verification misses where the agent did an inaccurate search or missed domain knowledge (wrongly claiming a paper did not implement its seven-seed robustness runs, and wrongly claiming a baseline was used with non-standard hyperparameters), and three were over-interpretations of correctly-read code (methodology/validity and technical-bug claims the code implemented, but which do not manifest under the authors’ protocol), all of low-to-medium severity. This analysis supports our conclusion from the robustness runs that hallucinations are not a central issue.

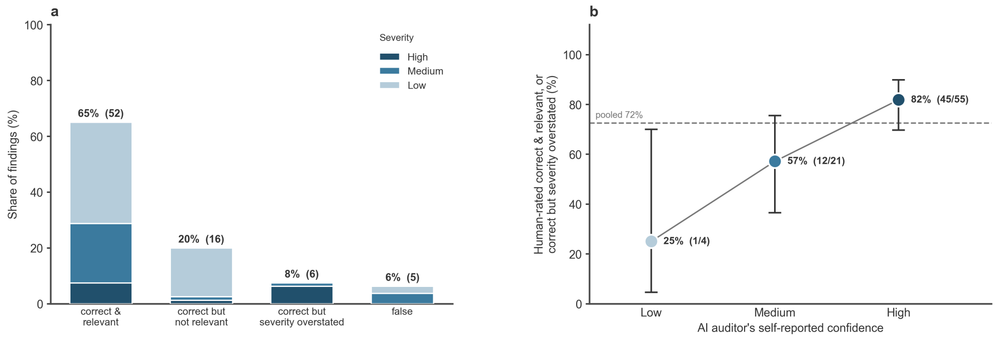


Figure 6: **Human evaluation of agentic auditors' findings. (a)** Human verdicts on all rated discrepancies (except for one finding, which was rated "unsure"). **(b)** Confidence calibration: the fraction of findings the human expert judged rated as correct and relevant (possibly with overstated severity) against the self-reported mode confidence of the agentic auditor. Error bars are Wilson 95% confidence intervals.

4.3 Token cost and run time

Our agentic workflow is feasible to run routinely. Across the 87 papers with retrieved code, each audit consumed a mean of 6.9M tokens, split between the main audit pass as well as the adversarial verification pass (Figure S2a). About 92% of tokens are cache reads, which most current APIs serve at a steep discount. Runtime is similarly manageable. A single paper takes a mean of 9.8 minutes to run through the audit as well as the verifier end-to-end (Figure S2b). As papers are audited in isolation with no cross-talk, we can parallelize running in batches bound only by the slowest single agent. We carried out the audits in batches of around 20 papers. The longest paper took 23.4 minutes. In total, we needed less than 100 minutes to audit all papers. Per paper, the auditor yields a median of 7 discrepancies (range between 2 and 13).

This means agentic code auditing is well within the budget of a pre-submission self-check or a reviewer-side screening, all at a fraction of the human time cost for a manual codebase audit.

5 Discussion

Our meta-audit on a subsample of NeurIPS 2025 papers reveals that the gap between papers' claims and their released code is large, systematic, but cheap to detect. The main/dominant discrepancy category was not methodological error but rather simple code absence (51% of the discrepancies), which does, however, make the other three discrepancy types undetectable. The average paper was missing 44% of the code backing the arguments our auditor deemed load-bearing. This was followed by paper-code mismatches which make up another 25%.

In the context of the literature on systematic reproducibility analyses, the NeurIPS papers we audit do not score well. For example, every audited paper fell short of Heil's reproducibility Heil et al. (2021), where the lowest tier (Bronze) requires code, data, and trained model weights to be deposited in an immutable, citable DOI archive (such as Zenodo) for permanent storage. This is, however, expected because NeurIPS submission guidelines do not mandate this currently.

Our numbers provide a first estimate for reproducibility in a community where these standards are currently self-reported without systematic peer review of the code. Furthermore, the discrepancy of roughly 39% of NeurIPS 2019 papers self-reporting code availability falling to around 28% once a reviewer checked code availability is evidence that author checklists may overstate the reality of code availability (Magnusson et al., 2023). Our pipeline goes beyond the trivial check of repository availability, which is a necessary but not sufficient standard and analyzes whether the code behind the link can substantiate the claims in a paper.

The gold standard to validate a computational paper's findings is to fully reproduce the tool by re-running the released code with the data and on new datasets. However, it commonly entails prohibitive compute cost, facing unreleased datasets, unrecoverable software environments, or external components that are hard to reproduce, and does not necessarily lead to the detection of methodological or mismatch discrepancies. Our contribution is deliberately a high-yield but low-cost audit that is complementary to the full reproduction approach. We do not attempt full reproduction, but rather audit traceability and correctness: i.e., that each headline claim maps to code capable of producing it with the same methodological steps described in the paper. Passing our code audit is a necessary condition for full reproducibility without having to re-implement the methods. This distinction is what makes our tool feasible at scale.

Running *AuditOwl* has a reasonable cost. The entire study (100 papers, 87 audits plus 50 robustness re-runs) was carried out on a single consumer Claude Max subscription rather than a metered enterprise account. Therefore, the pipeline is well-suited for deployment as a routine quality control tool. For example, authors can catch mismatches between their output writings and code, and journals or conferences can make code reviewable, without having to rely on more fully automated peer reviewing. Agentic code audit can transform code review from a difficult search problem in codebases to the validation of specific, possibly problematic line and file locations. *AuditOwl* is not designed to pass a verdict on a paper in the publishing process, but rather to assist authors and reviewers to address clear, objective issues in the code. The adversarial verification pass was designed to re-check every finding against the cited evidence to prevent hallucinated lines of code or text that would break the evidence trace. However, it only corrected 3 of 606 findings, two severity/confidence downgrades, one rejection of a false positive finding that conflated two experiments. Depending on the token budget, this step may be optional.

In our robustness experiments, when pooling independent audit reports numbers of distinct discrepancies saturates, which indicates that run-to-run variability is primarily a recall limitation rather than hallucinations: the auditor raises a different subset of true but borderline relevant discrepancies each time, instead of inventing false positives. This is confirmed by the overall low false positive rates in the human validation trial. Recall limitations of smaller issues can be simply mitigated by ensembling a small number of re-runs at a trade-off between increasing costs and recall rate. However, code can contain deeper issues that require in-depth domain knowledge, where ensembling cannot help to improve recovery.

6 Limitations

Agentic verification-centric code audit is an efficient method to increase reproducibility and trust in computer science publishing, however, there are limitations to our methodology. Our sample of 100 empirical papers from a single venue and year is not enough to generalize over the whole NeurIPS, let alone the machine learning publishing. The steep reproducibility funnel is, however, drastic enough for us to provide a qualitative conclusion that there is often little code evidence behind publications, even if factoring in the possible error rate of the agentic auditing.

We used Claude Opus 4.8 as the main model behind our approach, and we do not expect that our pipeline works well with much weaker models. We open-source our pipeline to enable use and performance evaluations in other settings as the model landscape behind agentic systems is rapidly evolving.

By design, we focus on auditing traceability, i.e., we locate claims and their code and check if it is implemented in a methodologically faithful way as described in the paper. We do not re-run all code to reproduce published numbers for all checks, but rather a subset, since full re-execution is unfeasible.

Reading PDFs is token-expensive and prone to loss for agents, and using automated text file creation from PDFs is not fully reliable. We input both PDFs and generated text files for the agent to consume; however, we cannot promise full input fidelity. Agentic code auditing can benefit from trends towards more machine-readable science (Booeshaghi et al., 2026). Furthermore, while we shallow-clone code repositories and pull LFS content, we do not retrieve large datasets residing outside the repository. We suspect this can create a ceiling for the recoverability of methodological problems (e.g., a simple baseline might outperform a proposed model, but this can only be tested with the dataset available).

Finally, agentic systems can make mistakes and are vulnerable to prompt injection. We imagine two classes of attack: an author might try to bias the agentic system toward a favorable audit, or try to run malicious code on the system that performs the audit. The latter can likely be mostly addressed by sandboxing; for the former and possible mistakes, a human reviewer must validate the audit before it is used to evaluate papers and detect suspicious patterns.

A natural objection to our work here is that we only human-validate a small subset of the discrepancies which were flagged: We argue that each audit should be validated, yet we report findings across 100 papers that are not all themselves exhaustively human-validated. We, however, only characterize distributional tendencies and never judge individual work. We critique the community-level standard where code availability and completeness is self-reported but not reviewed. In line with this, our conclusions are robust to the measured error rate: human validation of the 80 examined discrepancies yielded 74 correct, with false positives concentrated in low-severity discrepancies, which we exclude in most analyses and overall reproducibility assessment. Finally, a complete human validation of discrepancies across our full benchmark presented here is not within our scope, but validating the findings for a single paper is entirely tractable for that paper's author or an expert reviewer.

7 Ethics Statement

Our agentic code audit system is designed to augment human review, and by no means replace it. It produces a quote-anchored human-readable evidence trail and no verdict, steering attention to specific file and line locations. The output is formatted in a way that facilitates a human expert to check every finding. Not all discrepancies are decision-relevant (this also depends on community standards and journal guidelines), and using them to pass judgment might unfairly harm authors. Our audit has flagged flaws in real published work, but examples here serve only to illustrate categories and to investigate general trends and should not be used to judge researchers, especially since the issues we observe are systematic and not individual shortcomings. We also note that missing-code rates may correlate with lab resources, so discrepancy counts measure traceability, not necessarily contribution quality, and should not be repurposed as a direct reproducibility judgment. Our tool exists to make code review feasible and scale human expertise, never to replace it.

8 Conclusion

Code is the primary evidence underpinning computational results, yet it remains largely unexamined during the peer-review process. We introduced *AuditOwl*, an open-source agentic pipeline that traces a paper's headline claims to its released code, writes and runs deterministic checks in a sandbox, and re-verifies all findings through a further adversarial pass. Applied to 100 randomly sampled NeurIPS 2025 papers, it produced a graded and quote-anchored evidence trail in minutes for each paper, revealing a steep reproducibility funnel, where incomplete or mismatched code is a standard occurrence as opposed to a rare finding. *AuditOwl* provides a validation-centric approach to augment, instead of automating judgment. It provides a feasible pre-submission check for authors as well as a recommendation of which parts of the code to pay attention to for reviewers. Submission and codebase volumes surge in computer science. This can outgrow available human attention needed to verify them, and tools to make human verification more efficient are needed. Making agentic paper-to-code auditing a routine step in publishing is a low-cost, yet immediately actionable pathway towards more reproducible computational science.

9 Reproducibility Statement

Our pipeline splits into a deterministic part and a stochastic LLM-agent-based part. The deterministic part includes randomly selecting empirical papers from the NeurIPS 2025 main track, which we do with a fixed seed. The post-processing of the audits is also fully deterministic. The audit and verify passes depend on stochastic LLMs, which we run on third-party repositories that might drift over time (we cannot redistribute the third-party code we audit). To alleviate this, we publish all discrepancies and agentic audit reports on our GitHub alongside the code for pre- and post-processing and plotting. A full re-run costs about 509M tokens for the audit, 94M for the verification run, and roughly 25 minutes of time if run in parallel. Furthermore, we run *AuditOwl* for our contribution and append the resulting audit in Section S1.2.

References

Akhil Pandey Akella, Harish Varma Siravuri, and Shaurya Rohatgi. Pre-review to peer review: Pitfalls of automating reviews using large language models, 2025. URL https://arxiv.org/abs/2512.22145.

Anonymous. Formatting Instructions For NeurIPS 2025, 2025. URL https://arxiv.org/html/2506.15953v1.

Anonymous. Beyond Imitation: A Framework and Benchmark for LLM-Assisted Peer Review. *Transactions on Machine Learning Research*, April 2026. URL https://openreview.net/forum?id=7iX2Z2bPFB.

Samar Ansari. Compound Deception in Elite Peer Review: A Failure Mode Taxonomy of 100 Fabricated Citations at NeurIPS 2025, February 2026. URL http://arxiv.org/abs/2602.05930. arXiv:2602.05930 [cs.DL].

Simon Baker. Why did the Nature Index grow by 16% in 2024?, July 2025. URL https://www.nature.com/nature-index/news/why-did-the-nature-index-grow-by-sixteen-percent-in-twenty-twenty-four.

Federico Bianchi, Yongchan Kwon, Zachary Izzo, Linjun Zhang, and James Zou. *To Err Is Human: Systematic Quantification of Errors in Published AI Papers via LLM Analysis*. December 2025. doi: 10.48550/arXiv.2512.05925.

Joydeep Biswas, Sheila Schoepp, Gautham Vasan, Anthony Opipari, Arthur Zhang, Zichao Hu, Sebastian Joseph, Matthew Lease, Junyi Jessy Li, Peter Stone, Kiri L. Wagstaff, Matthew E. Taylor, and Odest Chadwicke Jenkins. AI-Assisted Peer Review at Scale: The AAAI-26 AI Review Pilot, April 2026. URL http://arxiv.org/abs/2604.13940. arXiv:2604.13940 [cs.AI].

A. Sina Booeshaghi, Laura Luebbert, and Lior Pachter. Science should be machine-readable. *bioRxiv*, 2026. doi: 10.64898/2026.01.30.702911. URL https://www.biorxiv.org/content/early/2026/02/02/2026.01.30.702911.

Odd Erik Gundersen. The fundamental principles of reproducibility. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 379(2197):20200210, March 2021. ISSN 1364-503X. doi: 10.1098/rsta.2020.0210. URL https://doi.org/10.1098/rsta.2020.0210.

Odd Erik Gundersen and Sigbjørn Kjensmo. State of the Art: Reproducibility in Artificial Intelligence. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), April 2018. ISSN 2374-3468. doi: 10.1609/aaai.v32i1.11503. URL https://ojs.aaai.org/index.php/AAAI/article/view/11503.

Benjamin Haibe-Kains, George Alexandru Adam, Ahmed Hosny, Farnoosh Khodakarami, Levi Waldron, Bo Wang, Chris McIntosh, Anna Goldenberg, Anshul Kundaje, Casey S. Greene, Tamara Broderick, Michael M. Hoffman, Jeffrey T. Leek, Keegan Korthauer, Wolfgang Huber, Alvis Brazma, Joelle Pineau, Robert Tibshirani, Trevor Hastie, John P. A. Ioannidis, John Quackenbush, and Hugo J. W. L. Aerts. Transparency and reproducibility in artificial intelligence. *Nature*, 586(7829):E14–E16, October 2020. ISSN 1476-4687. doi: 10.1038/s41586-020-2766-y. URL https://www.nature.com/articles/s41586-020-2766-y.

Benjamin J. Heil, Michael M. Hoffman, Florian Markowetz, Su-In Lee, Casey S. Greene, and Stephanie C. Hicks. Reproducibility standards for machine learning in the life sciences. *Nature Methods*, 18(10):1132–1135, October 2021. ISSN 1548-7105. doi: 10.1038/s41592-021-01256-7. URL https://www.nature.com/articles/s41592-021-01256-7.

Matthew Hutson. Artificial intelligence faces reproducibility crisis. *Science*, 359(6377):725–726, February 2018. ISSN 1095-9203. doi: 10.1126/science.359.6377.725.

Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, September 2023. ISSN 2666-3899. doi: 10.1016/j.patter.2023.100804.

Sayash Kapoor, Emily M. Cantrell, Kenny Peng, Thanh Hien Pham, Christopher A. Bail, Odd Erik Gundersen, Jake M. Hofman, Jessica Hullman, Michael A. Lones, Momin M. Malik, Priyanka Nanayakkara, Russell A. Poldrack, Inioluwa Deborah Raji, Michael Roberts, Matthew J. Salganik, Marta Serra-Garcia, Brandon M. Stewart, Gilles Vandewiele, and Arvind Narayanan. REFORMS: Consensus-based Recommendations for Machine-learning-based Science. *Science Advances*, 10(18):eadk3452, May 2024. doi: 10.1126/sciadv.adk3452. URL https://www.science.org/doi/10.1126/sciadv.adk3452.

Seungone Kim, Dongkeun Yoon, Kiril Gashteovski, Juyoung Suk, Jinheon Baek, Pranjal Aggarwal, Ian Wu, Viktor Zaverkin, Spase Petkoski, Daniel R. Schrider, Ilija Dukovski, Francesco Santini, Biljana Mitreska, Yong Jeong, Kyeongha Kwon, Young Min Sim, Dragana Manasova, Arthur Porto, Biljana Mojsoska, Makoto Takamoto, Marko Shuntov, Ruoqi Liu, Hyunjoo Jenny Lee, Niyazi Ulas Dinç, Yehhyun Jo, Sunkyu Han, Chungwoo Lee, Huishan Li, Esther H. R. Tsai, Ergun Simsek, Khushboo Shafi, Yeonseung Chung, Jihye Park, Aleksandar Shulevski, Henrik Christiansen, Yoosang Son, Elly Knight, Amanda Montoya, Jeongyoun Ahn, Christian Langkammer, Heera Moon, Changwon Yoon, Nikola Stikov, Mooseok Jang, Edward Choi, Junhan Kim, Yeon Sik Jung, Woo Youn Kim, Jae Kyoung Kim, Ishraq Md Anjum, Hyun Uk Kim, Drew Bridges, Carolin Lawrence, Xiang Yue, Alice Oh, Akari Asai, Sean Welleck, and Graham Neubig. On the limits and opportunities of ai reviewers: Reviewing the reviews of nature-family papers with 45 expert scientists, 2026. URL https://arxiv.org/abs/2605.20668.

Ian Magnusson, Noah A. Smith, and Jesse Dodge. Reproducibility in NLP: What have we learned from the checklist? In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 12789–12811, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.809. URL https://aclanthology.org/2023.findings-acl.809/.

Joelle Pineau, Koustuv Sinha, Genevieve Fried, Rosemary Nan Ke, and Hugo Larochelle. ICLR Reproducibility Challenge 2019. *ReScience C*, 5(2):#5, May 2019. doi: 10.5281/zenodo.3158244. URL https://zenodo.org/records/3158244.

Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *The Journal of Machine Learning Research*, 22(1): 164:7459–164:7478, January 2021. ISSN 1532-4435. URL https://dl.acm.org/doi/10.5555/3546258.3546422.

Edward Raff. A step toward quantifying independently reproducible machine learning research. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/c429429bf1f2af051f2021dc92a8ebea-Paper.pdf.

Marta Serra-Garcia and Uri Gneezy. Nonreplicable publications are cited more than replicable ones. *Science Advances*, 7(21):eabd1705, 2021. doi: 10.1126/sciadv.abd1705.

Zachary S. Siegel, Sayash Kapoor, Nitya Nagdir, Benedikt Stroebl, and Arvind Narayanan. Core-bench: Fostering the credibility of published research through a computational reproducibility agent benchmark, 2024. URL https://arxiv.org/abs/2409.11363.

Koustuv Sinha, Jesse Dodge, Sasha Luccioni, Jessica Zosa Forde, Sharath Chandra Raparthy, Joelle Pineau, and Robert Stojnic. Ml reproducibility challenge 2021. *ReScience C*, 8(2), May 2022. doi: 10.5281/zenodo.6574723. URL https://doi.org/10.5281/zenodo.6574723.

Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. Paperbench: Evaluating ai's ability to replicate ai research, 2025. URL https://arxiv.org/abs/2504.01848.

Chenhao Tan and Haokun Liu. The Mirage of Autonomous AI Scientists. February 2026. URL https://chenhaot.com/papers/mirage_ai_scientist.pdf.

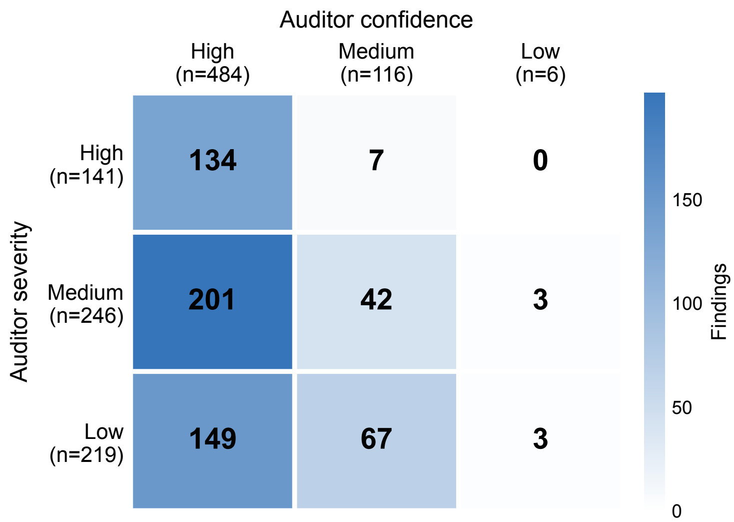
Varad Vishwarupe, Nigel Shadbolt, Marina Jirotka, and Ivan Flechais. Neurips should require reproducibility standards for frontier ai safety claims, May 2026. URL `https://arxiv.org/abs/2605.08192`.

Shuo Yan, Ruochen Li, Ziming Luo, Zimu Wang, Daoyang Li, Liqiang Jing, Kaiyu He, Peilin Wu, George Michalopoulos, Yue Zhang, Ziyang Zhang, Mian Zhang, Zhiyu Chen, and Xinya Du. Lmr-bench: Evaluating llm agent's ability on reproducing language modeling research, 2025. URL `https://arxiv.org/abs/2506.17335`.

Sungduk Yu, Man Luo, Avinash Madasu, Vasudev Lal, and Phillip Howard. Is your paper being reviewed by an llm? investigating ai text detectability in peer review, 2024. URL `https://arxiv.org/abs/2410.03019`.

A Appendix

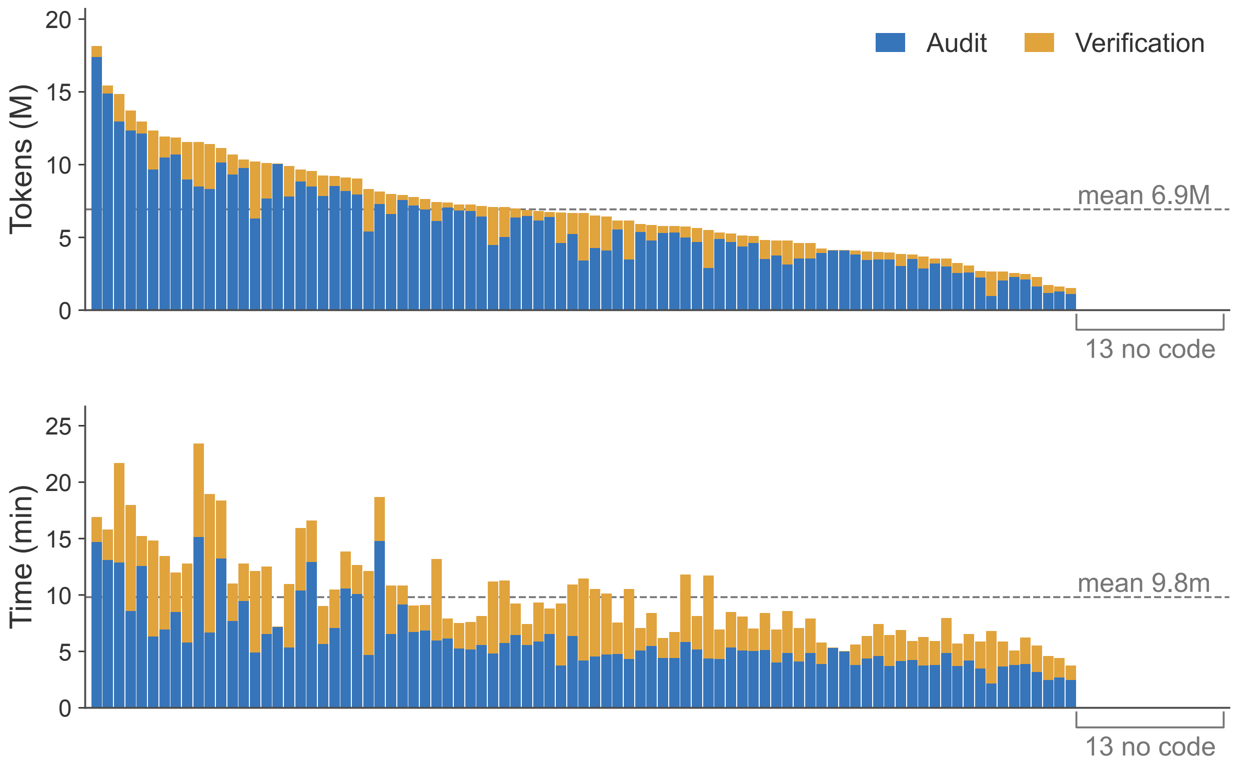
S1.1 Supplementary Figures



Supplementary Figure S1: **Number of flagged discrepancies by severity and confidence.** For the 606 discrepancies found in the 87 papers that released their own source code, most are flagged with medium severity at high confidence. Note both axes are the agent's own labels.

S1.2 Supplementary Information - Audit for this contribution

TBA



Supplementary Figure S2: **Effort per audit.** Per-paper cost and runtime for 100 papers (87 run audits, as 13 had no retrievable code). **(a)** Tokens per paper, split into the audit pass (blue), and the adversarial verification pass (orange); the mean is 6.9M tokens per paper. Across the full batch, total consumption is around 603M tokens, where approximately 94M are the agentic verifier. **(b)** Wall-clock time per paper, mean 9.8 min end-to-end (including Opus audit, Sonnet verification, Opus escalation). As the agents run in isolation, audits can run cleanly in parallel and only take as long as the longest paper requires. In our papers, the longest audit was approximately 23.4 minutes.